



Optimización Bayesiana para Variables Mixtas con Restricciones de Caja Negra

Proyecto Final – Optimización

Lucas Miguel Iturriago Salas

Universidad Nacional de Colombia
Departamento de Ingeniería Eléctrica, Electrónica y Computación

Primer Semestre - 2026

Formulación del Problema: ¿Qué buscamos?



En ingeniería, optimizar no es solo "mejorar", es encontrar el balance perfecto bajo restricciones físicas de hardware.

1. La Meta (Maximizar)

$$\max_{x \in \mathcal{X}} f(x)$$

Métrica de Rendimiento:

- Validation DICE Score $\in [0, 1]$.
- Mide el solapamiento exacto de la segmentación de la U-Net.

+

2. El Límite (Restringir)

$$g(x) \leq 0$$

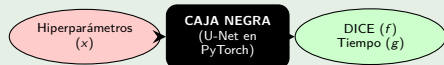
Restricción Física:

- $T(x) - T_{\text{máx}} \leq 0$
- Donde $T_{\text{máx}} = 35,0$ segundos por época (Límite en CPU).

$\mathcal{X}$ = Espacio de búsqueda mixto de 3 dimensiones (*Learning Rate*, α_{focal} , Optimizador).

¿Por qué no podemos usar Métodos Analíticos o Descenso de Gradiente clásico ($\nabla f(x)$)?

La Red Neuronal como Pared



El sistema recibe un vector continuo-discreto, pero la transición matemática interna es oculta y costosa.

- **No es Diferenciable:** No existe una ecuación para $\nabla f(x)$. No hay derivadas.
- **Evaluación Costosa:** Cada “pregunta” al sistema cuesta ~ 30 segundos de cómputo.
- **Presencia de Ruido:** El mismo punto x puede variar su tiempo ligeramente por la CPU.

Consecuencia

Estamos obligados a adivinar el comportamiento global usando la menor cantidad de experimentos posibles.

La Optimización Bayesiana no adivina a ciegas; utiliza la filosofía de Thomas Bayes para adaptar un modelo matemático a medida que PyTorch devuelve resultados reales.

El Teorema de Bayes para Funciones

$$P(f | D) = \frac{P(D | f) \times P(f)}{P(D)}$$

1. El Prior $P(f)$

Nuestra creencia inicial sobre la función (ej. asumir que el DICE Score es suave mediante el Kernel Matérn 5/2).

×

2. La Verosimilitud $P(D | f)$

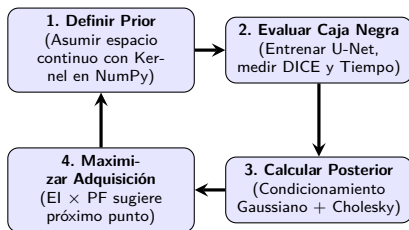
El baño de realidad. La probabilidad de observar los DICE Scores y tiempos reales medidos en PyTorch.

→

3. El Posterior $P(f | D)$

La combinación de ambos mundos. Nuestro modelo actualizado con media y varianza en cada punto.

Este enfoque da origen a un ciclo de aprendizaje activo dentro de tu código:



Ventaja de Ingeniería

La verosimilitud actúa como un **filtro y un imán**: descarta funciones del prior alejadas de los datos reales y refuerza regiones de alto DICE Score.

¿Qué es un Proceso Gaussiano? (Intuición Detrás del Nombre)

Antes de ver las ecuaciones, entendamos los dos pilares conceptuales que le dan nombre a esta herramienta matemática:

1. ¿Por qué es un PROCESO?

Porque no modela un único punto, sino una **colección infinita de variables aleatorias** indexadas de forma continua a lo largo del espacio de hiperparámetros $\mathcal{X}$.

- Imagina un **"hilo infinito"** de posibles funciones que representan el DICE Score.

2. ¿Por qué es GAUSSIANO?

Porque si tomas **cualquier rebanada o subconjunto finito de n puntos** de ese espacio para evaluarlos, esos n puntos se distribuirán conjuntamente como una **Distribución Normal Multivariada** de n dimensiones.

$$\mathbf{f} \sim \mathcal{N}(\mathbf{0}, K(\mathbf{X}, \mathbf{X}))$$

¿Cómo sabe el Proceso Gaussiano qué tan parecidos son dos puntos en el espacio si nunca los ha evaluado? A través de la **Función de Covarianza o Kernel**:

Ecuación del Kernel Matérn 5/2

$$k(x, x') = \sigma_f^2 \left(1 + \sqrt{5}d(x, x') + \frac{5}{3}d(x, x')^2 \right) \exp \left(-\sqrt{5}d(x, x') \right)$$

- $d(x, x') = \sqrt{\sum \frac{(x_i - x'_i)^2}{\ell_i^2}}$: Distancia euclidiana ponderada por la longitud de escala ℓ_i de cada hiperparámetro.
- σ_f^2 (**Varianza de la Señal**): Controla qué tan vertical u oscilatoria puede llegar a ser la función objetivo.

Intuición Didáctica

Si dos configuraciones están cerca en el espacio ($d \rightarrow 0$), el kernel devuelve una covarianza alta ($\approx \sigma_f^2$). El GP asume que si un *learning rate* funcionó bien, sus vecinos inmediatos también lo harán.



El entrenamiento de una red neuronal en hardware es intrínsecamente estocástico (ruidoso). Si evaluamos dos veces exactamente los mismos hiperparámetros, el tiempo de CPU y el DICE Score variarán ligeramente.

Para modelar esta realidad de ingeniería, añadimos un término de **Ruido Gaussiano Blanco Homocedástico** ($\epsilon \sim \mathcal{N}(0, \sigma_n^2)$) directo a nuestras observaciones reales de PyTorch:

$$y = f(x) + \epsilon$$

Impacto Algebraico en la Diagonal de la Covarianza

En nuestras matrices de NumPy, esto se traduce en sumarle un pequeño valor a la diagonal principal, creando la matriz ruidosa de observaciones K_y :

$$K_y = K(X, X) + \sigma_n^2 I$$

Esto no solo modela la incertidumbre real de la red, sino que actúa como regularizador numérico para evitar que la matriz sea singular.



¡Aquí está el puente! Si tenemos n observaciones pasadas $\mathbf{y}$ obtenidas en PyTorch y queremos predecir el rendimiento teórico f_* en un nuevo punto candidato x_* , la definición del GP nos obliga a empaquetarlos en una **Distribución Conjunta Gaussiana de dimensión** $(n + 1)$:

Modelado Conjunto Estricto (Prior + Verosimilitud)

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ 0 \end{bmatrix}, \begin{bmatrix} K(X, X) + \sigma_n^2 I & K(X, x_*) \\ K(x_*, X) & k(x_*, x_*) \end{bmatrix} \right)$$

- $K_y = K(X, X) + \sigma_n^2 I$: Covarianza de los datos conocidos con ruido $(n \times n)$.
- $K_* = K(X, x_*)$: Vector de correlación entre datos históricos y el nuevo candidato $(n \times 1)$.
- $k_* = k(x_*, x_*)$: Varianza intrínseca del punto candidato (1×1) .

Condicionamiento: De la Matriz Conjunta a las Predicciones

Al aplicar el teorema de **Condicionamiento Gaussiano** sobre la gran matriz particionada de la diapositiva anterior, la distribución conjunta colapsa en el **Posterior** analítico:

Media Posterior (μ_*) $\rightarrow$ Estimación del Rendimiento

$$\mu(x_*) = K_*^T K_y^{-1} \mathbf{y}$$

Varianza Posterior (σ_*^2) $\rightarrow$ Cuantificación del Riesgo/Incertidumbre

$$\sigma^2(x_*) = k_* - K_*^T K_y^{-1} K_*$$

Bajo la hipótesis de independencia estadística entre el rendimiento y el tiempo, la teoría de utilidad condicional dicta una regla de decisión multiplicativa:

Función de Adquisición Restringida $\alpha_c(x)$

$$\alpha_c(x) = EI(x) \times PF(x)$$

1. Rendimiento: $EI(x)$

Mejora Esperada:

- Maximiza el DICE Score.
- Estima la ganancia potencial sobre el récord factible actual $f(x^+)$.

2. Viabilidad: $PF(x)$

Escudo de Hardware:

- Probabilidad de que $T(x) \leq T_{\text{máx}}$.
- Modelado con la CDF normal:

$$PF(x) = \Phi\left(\frac{T_{\text{máx}} - \mu_T(x)}{\sigma_T(x)}\right)$$

Para resolver analíticamente la viabilidad de hardware y la mejora esperada se puede recurrir a las dos funciones fundamentales de la distribución Gaussiana estándar $\mathcal{N}(0, 1)$:

PDF: Función de Densidad ($\phi(Z)$)

$$\phi(Z) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{Z^2}{2}\right)$$

- Representa la altura de la campana de Gauss en la coordenada Z .
- **Rol en el algoritmo:** Modela el potencial de **exploración** pura midiendo el peso de la incertidumbre.

CDF: Función Acumulada ($\Phi(Z)$)

$$\Phi(Z) = \int_{-\infty}^Z \phi(t) dt$$

- Representa el área bajo la curva; la probabilidad de que un evento sea $\leq Z$.
- **Rol en el algoritmo:** Modela la **viabilidad** en el tiempo (PF) y la fuerza de **explotación** en el rendimiento (EI).

Expected Improvement (EI) para Maximización

Para guiar la búsqueda hacia el máximo global de la función objetivo, calculamos la esperanza matemática exacta de la mejora bajo la distribución normal del posterior:

Integral Analítica de EI

$$EI(x) = \int_{f(x^+)}^{\infty} (f - f(x^+)) \mathcal{N}(f; \mu(x), \sigma^2(x)) df$$

$$EI(x) = (\mu(x) - f(x^+) - \xi) \Phi(Z) + \sigma(x) \phi(Z) \quad \text{donde } Z = \frac{\mu(x) - f(x^+) - \xi}{\sigma(x)}$$

- **Término Izquierdo (Explotación):** Ponderado por la CDF $\Phi(Z)$. Prioriza regiones donde la media posterior $\mu(x)$ es superior al mejor valor factible actual $f(x^+)$, cuantificando el valor esperado de dicha ganancia.
- **Término Derecho (Exploración):** Ponderado por la PDF $\phi(Z)$. Premia al algoritmo por explorar zonas de alta varianza o incertidumbre $\sigma(x)$, donde la densidad de probabilidad aún está dispersa.
- $\xi = 0,01$ (**Parámetro de Exploración**): Desplazamiento numérico en Z que regula el balance entre ambos componentes, forzando un umbral mínimo de mejora.

- **Rigor Matemático Descentralizado:** Se construyó un framework completo continuo-discreto con restricciones desde cero en NumPy, evitando librerías de caja negra tradicionales.
- **Estabilidad Numérica Blindada:** La integración de ruidos homocedásticos diferenciados y la factorización de Cholesky protegieron el cómputo de singularidades matriciales.
- **Ingeniería Eficiente:** El modelo matemático logró mapear la topología del hardware de forma autónoma, deteniendo la exploración en zonas infactibles por tiempo y maximizando el DICE Score de la U-Net en las regiones viables.

¡Muchas gracias por su atención!
¿Preguntas o comentarios?